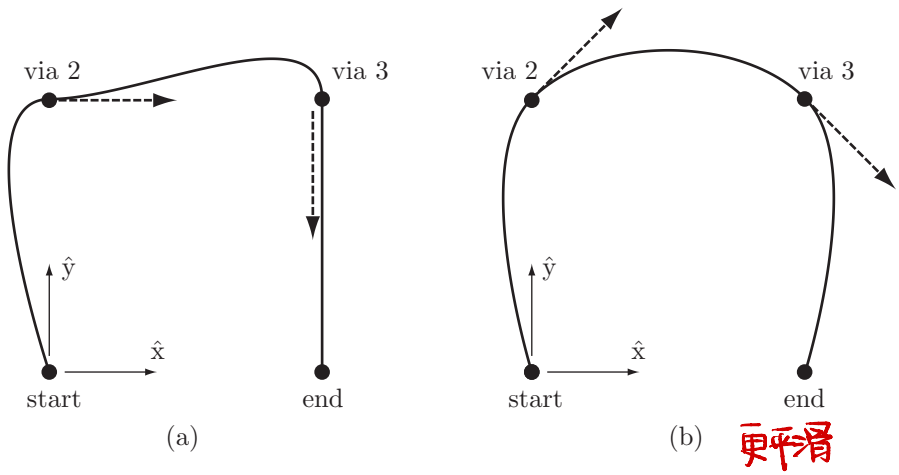


**Figure 9.7:** Two paths in an $(x, y)$ space corresponding to piecewise-cubic trajectories interpolating four via points, including a start point and an end point. The velocities at the start and end are zero, and the velocities at vias 2 and 3 are indicated by the dashed tangent vectors. The shape of the path depends on the velocities specified at the via points.

## 9.3 Polynomial Via Point Trajectories

多项式差值 直接一次求出θ(t)

If the goal is to have the robot joints pass through a series of **via points** at specified times, without a strict specification on the shape of path between consecutive points, a simple solution is to use polynomial interpolation to find joint histories $\theta(t)$ directly without first specifying a path $\theta(s)$ and then a time scaling $s(t)$ (Figure 9.7).

不用先求path
再time scaling

Let the trajectory be specified by $k$ via points, with the start point occurring at $T_1 = 0$ and the final point at $T_k = T$. Since each joint history is interpolated individually, we focus on a single joint variable and call it $\beta$ to avoid a proliferation of subscripts. At each via point $i \in \{1, \ldots, k\}$, the user specifies the desired position $\beta(T_i) = \beta_i$ and velocity $\dot{\beta}(T_i) = \dot{\beta}_i$. The trajectory has $k-1$ segments, and the duration of segment $j \in \{1, \ldots, k-1\}$ is $\Delta T_j = T_{j+1} - T_j$. The joint trajectory during segment $j$ is expressed as the third-order polynomial

位置/速度
用户指定

第j段 关节轨迹

$$\beta(T_j + \Delta t) = a_{j0} + a_{j1}\Delta t + a_{j2}\Delta t^2 + a_{j3}\Delta t^3 \tag{9.25}$$

4个系数

in terms of the time $\Delta t$ elapsed in segment $j$, where $0 \le \Delta t \le \Delta T_j$. Segment $j$

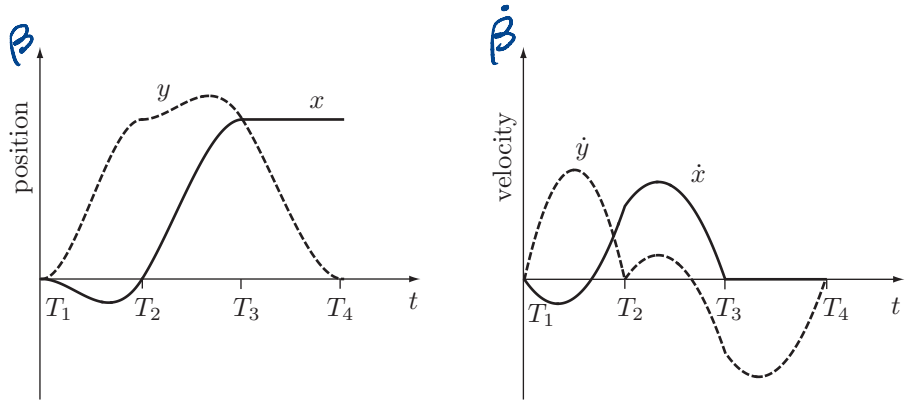


**Figure 9.8:** The coordinate time histories for the cubic via-point interpolation of Figure 9.7(a).

is subject to the four constraints

第j段

$$\begin{aligned} &\text{起点位置}\ \beta(T_j) = \beta_j, \qquad &&\text{起点速度}\ \dot{\beta}(T_j) = \dot{\beta}_j, \\ &\text{终点位置}\ \beta(T_j + \Delta T_j) = \beta_{j+1}, \qquad &&\text{终点速度}\ \dot{\beta}(T_j + \Delta T_j) = \dot{\beta}_{j+1}. \end{aligned}$$

4个条件确定三次多项式 4个系数

Solving these constraints for $a_{j0}, \ldots, a_{j3}$ yields

起点位置 $$a_{j0} = \beta_j, \tag{9.26}$$

起点速度 $$a_{j1} = \dot{\beta}_j, \tag{9.27}$$

加速度一半 $$a_{j2} = \frac{3\beta_{j+1} - 3\beta_j - 2\dot{\beta}_j \Delta T_j - \dot{\beta}_{j+1}\Delta T_j}{\Delta T_j^2}, \tag{9.28}$$

Jerk的 $\frac{1}{6}$ $$a_{j3} = \frac{2\beta_j + (\dot{\beta}_j + \dot{\beta}_{j+1})\Delta T_j - 2\beta_{j+1}}{\Delta T_j^3}. \tag{9.29}$$

Figure 9.8 shows the time histories for the interpolation of Figure 9.7(a). In this two-dimensional $(x, y)$ coordinate space the via points $1, \ldots, 4$ occur at times $T_1 = 0$, $T_2 = 1$, $T_3 = 2$, and $T_4 = 3$. The via points are at $(0, 0)$, $(0, 1)$, $(1, 1)$, and $(1, 0)$ with velocities $(0, 0)$, $(1, 0)$, $(0, -1)$, and $(0, 0)$.

Two issues are worth mentioning:

① 如只指定位置，可用启发式选速度 ↓ 路径更圆滑

- The quality of the interpolated trajectories is improved by "reasonable" combinations of via-point times and via-point velocities. For example, if the user wants to specify a via-point location and time, but not the velocity, a heuristic could be used to choose a via velocity on the basis of

the times and coordinate vectors to the via points before and after the via in question. As an example, the trajectory of Figure 9.7(b) is smoother than the trajectory of Figure 9.7(a).

② 五次多项式

- Cubic via-point interpolation ensures that velocities are continuous at via points, but not accelerations. The approach is easily generalized to the use of fifth-order polynomials and specification of the accelerations at the via points, at the cost of increased complexity of the solution.

If only two points are used (the start and end point), and the velocities at each are zero, the resulting trajectory is identical to the straight-line cubic polynomial time-scaled trajectory discussed in Section 9.2.2.1.

There are many other methods for interpolating a set of via points. For example, B-spline interpolation is popular. In B-spline interpolation, the path may not pass exactly through the via points, but the path is guaranteed to be confined to the convex hull of the via points, unlike the paths in Figure 9.7. This can be important to ensure that joint limits or workspace obstacles are respected.

(Hermite 插值)
上述：多项式插值
保证精确穿过每个点
但容易摆动/过冲

B样条插值
不保证精确
但保证路径始终被包在中间点
构成的凸包内部
(避障) 安全、平滑

## 9.4 Time-Optimal Time Scaling

In the case where the path $\theta(s)$ is fully specified by the task or an obstacle-avoiding path planner (e.g., Figure 9.9), the trajectory planning problem reduces to finding a time scaling $s(t)$. One could choose the time scaling to minimize the energy consumed while meeting a time constraint, or to prevent spilling a glass of water that the robot is carrying. One of the most useful time scalings minimizes the time of motion along the path, subject to the robot's actuator limits. Such time-optimal trajectories maximize the robot's productivity.

While the trapezoidal time scalings of Section 9.2.2.2 can yield time-optimal trajectories, this is only under the assumption of straight-line motions, constant maximum acceleration $a$, and constant maximum coasting velocity $v$. For most robots, because of state-dependent joint actuator limits and the state-dependent dynamics

$$M(\theta)\ddot{\theta} + c(\theta, \dot{\theta}) + g(\theta) = \tau, \tag{9.30}$$

the maximum available velocities and accelerations change along the path.

In this section we consider the problem of finding the fastest possible time scaling $s(t)$ that respects the robot's actuator limits. We write the limits on the $i$th actuator as

$$\tau_i^{\min}(\theta, \dot{\theta}) \leq \tau_i \leq \tau_i^{\max}(\theta, \dot{\theta}). \tag{9.31}$$